

Security Review

Cantina Managed

PLUME NETWORK: NEST-CONTRACTS

Cantina Managed review by:

Jiri123, Security Researcher



September 16, 2026



Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
3	Findings	4
3.1	Medium Risk	4
3.1.1	Legacy teller deposit validates receiver then mints to the adapter	4
3.1.2	Refund path calls an optional fee selector that four chains do not expose	4
3.1.3	Bundler wrapper is the compliance subject instead of the calling wallet	4
3.2	Low Risk	5
3.2.1	Compliance proxy accepts an unvalidated vault and allows spoofed events	5
3.3	Informational	5
3.3.1	Composer kill switch blocked by metadata getters	5



1 Introduction

1.1 About Cantina

Cantina is a security platform that pairs a world-class network of security researchers with agentic AI to help teams building onchain find and fix real risk. Through managed reviews, competitions, and bounties, Cantina secures protocols before vulnerabilities can be exploited.

Learn more at cantina.security

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).



2 Security Review Summary

Plume is a public, EVM-compatible blockchain built for the next evolution of Real World Assets (RWAs).

From Aug 25th to Aug 31st the Cantina team conducted a review of `nest-contracts` on commit hash `90655ef2`. The team identified a total of **5** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	3	3	0
Low Risk	1	0	1
Gas Optimizations	0	0	0
Informational	1	1	0
Total	5	4	1

Final reviewed source snapshot, including verified fixes: `nest-contracts` on commit hash `c7f56325`. The corresponding `contracts/` git tree commit hash is `34a6abb8`, allowing independent verification that a public release contains identical directory contents. The review scope remains as defined in this report.



3 Findings

3.1 Medium Risk

3.1.1 Legacy teller deposit validates receiver then mints to the adapter

Severity: Medium Risk.

Context: [NestAdapter.sol#L492](#).

Description: `tellerPredicateDeposit()` checks `receiver` non-zero and then never uses it. The three-argument `teller.deposit()` overload has no recipient, so the teller mints to the adapter, and anyone can redeem those shares afterwards because `isOwnerOrOperator` accepts `address(this)` as owner and `instantRedeem` is public. Generated bundles spend the shares in the same transaction, so the loss hits a direct call that names a wallet.

Recommendation: Forward the minted shares to `receiver` before returning, or use `bulkDeposit(depositAsset, assets, minimumMint, receiver)`.

Plume Network: Fixed in commit [5a3b9888](#).

Cantina Managed: `tellerPredicateDeposit()` now forwards the minted shares to `receiver`, so shares are not left redeemable on the adapter.

3.1.2 Refund path calls an optional fee selector that four chains do not expose

Severity: Medium Risk.

Context: [BaseCCTPRelay.sol#L324](#).

Description: `_refund()` quotes `maxFee` with `TOKEN_MESSENGER.getMinFeeAmount(_amount)` on every chain, but only the Plume fee switch messenger exposes it. On Ethereum, Avalanche, Base and World Chain the call reverts while decoding the return value, so both refund branches in `_relay()` fail and the destination mint rolls back while the source burn stands.

Recommendation: Probe the selector as `DeployAndSetup.s.sol` already does, and fall back to `maxFee = 0` when it is missing.

Plume Network: Fixed in commit [50feccea](#).

Cantina Managed: Commit [50feccea](#) fixes this by having the relayer probe `getMinFeeAmount` and fall back to a zero fee floor on chains whose `TokenMessenger` does not expose the selector.

3.1.3 Bundler wrapper is the compliance subject instead of the calling wallet

Severity: Medium Risk.

Context: [NestBundler.sol#L248-L250](#).

Description: `getBundleAndExecute()` overwrites `ctx.initiator` with the wrapper before calling `Bundler3.multicall()`, so the adapter's compliance legs screen `NestBundler` instead of the calling wallet. The signed statement then carries no per-user field, `msgSender` is the fixed wrapper address and the payload is `accessCheck(address NestBundler)`, so one proof passes for any caller of `getBundleAndExecute()` and the single-use UUID bounds only how often a proof is spent, not by whom.

Recommendation: Bind the calling wallet as the compliance subject, for example by threading `bundle.ctx.owner` into the adapter's compliance legs.

Plume Network: Fixed in commit [3dc3ba1a](#).



Cantina Managed: Commits [3dc3ba1a](#) and [dd28e162](#) address this. `NestBundler` is necessarily the `Bundler3` initiator because it calls `multicall`, but generated compliance calls encode `msg.sender` as `onBehalf`. The signed statement therefore binds both `NestBundler` and the represented wallet. For redemptions, `owner` identifies the account whose shares are redeemed and `onBehalf` identifies the represented user for compliance. Note that with `legacyDeposit = true`, `tellerPredicateDeposit()` still validates the shared `NestBundler` rather than binding the calling wallet — this is a known limitation of Predicate V1 that will be removed when the route is deprecated.

3.2 Low Risk

3.2.1 Compliance proxy accepts an unvalidated vault and allows spoofed events

Severity: Low Risk.

Context: [ComplianceProxy.sol#L411](#).

Description: `_vault` comes from calldata and is never validated, so `_deposit()` approves and calls an arbitrary address and emits `Deposit` with caller chosen fields. No funds are at risk since the attacker round trips their own tokens, but the event comes from a trusted role holding contract, so any indexer or points pipeline reading it can be fed fabricated records.

Recommendation: Validate `_vault` against a governance set allowlist in every entrypoint.

Plume Network: Acknowledged.

Cantina Managed: Acknowledged. Spoofing requires valid compliance data and poses no direct fund risk. Indexers should filter on known vault addresses; adding a governance allowlist would increase configuration and integration risk, particularly given the proxy is owned by the timelock.

3.3 Informational

3.3.1 Composer kill switch blocked by metadata getters

Severity: Informational.

Context: [NestCCTPRelay.sol#L151-L153](#).

Description: `setComposer()` calls `ASSET_OFT()` and `ASSET_ERC20()` before the allowlist write, on the disable path as well as the enable path. A composer that reverts on either getter can no longer be turned off and keeps both its `isComposer` bit and its max USDC approval, which the `README` calls the per-path kill switch.

Recommendation: Move both checks inside the `_enabled` branch.

Plume Network: Fixed in commit [386e237c](#).

Cantina Managed: Commit [386e237c](#) fixes this by moving composer metadata validation so it runs only when enabling a composer. Disabling now always clears `isComposer` and revokes the USDC allowance, even if the composer's getters revert.